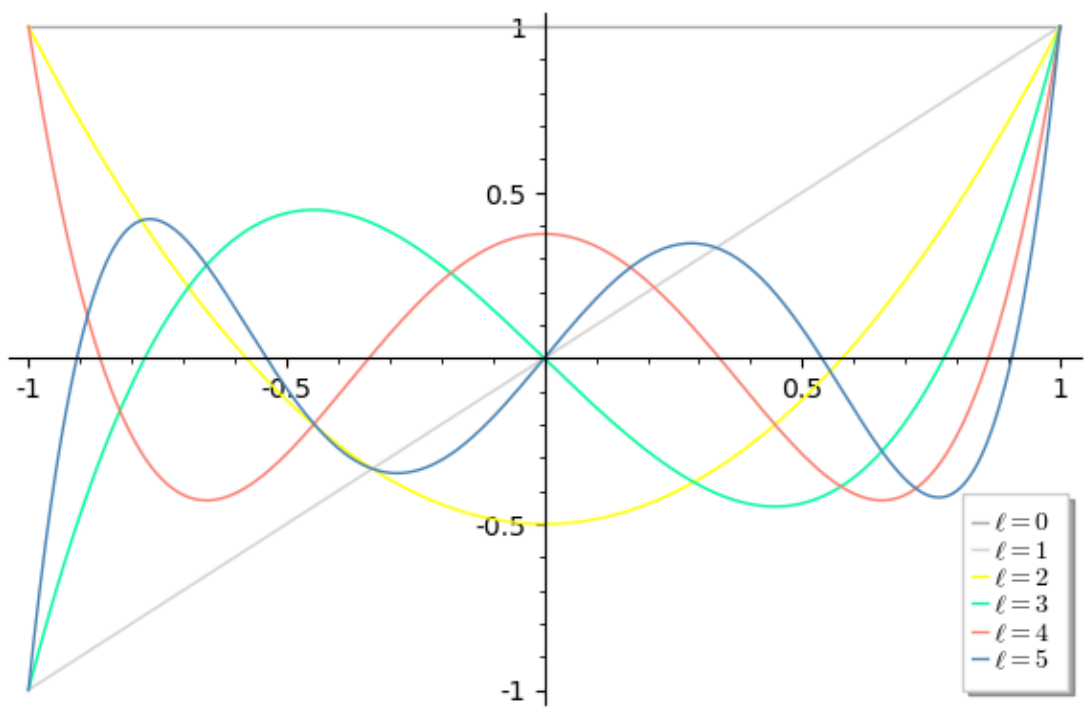
3160H A2

March 25, 2026

[2]:
```
import random
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = "all"
```

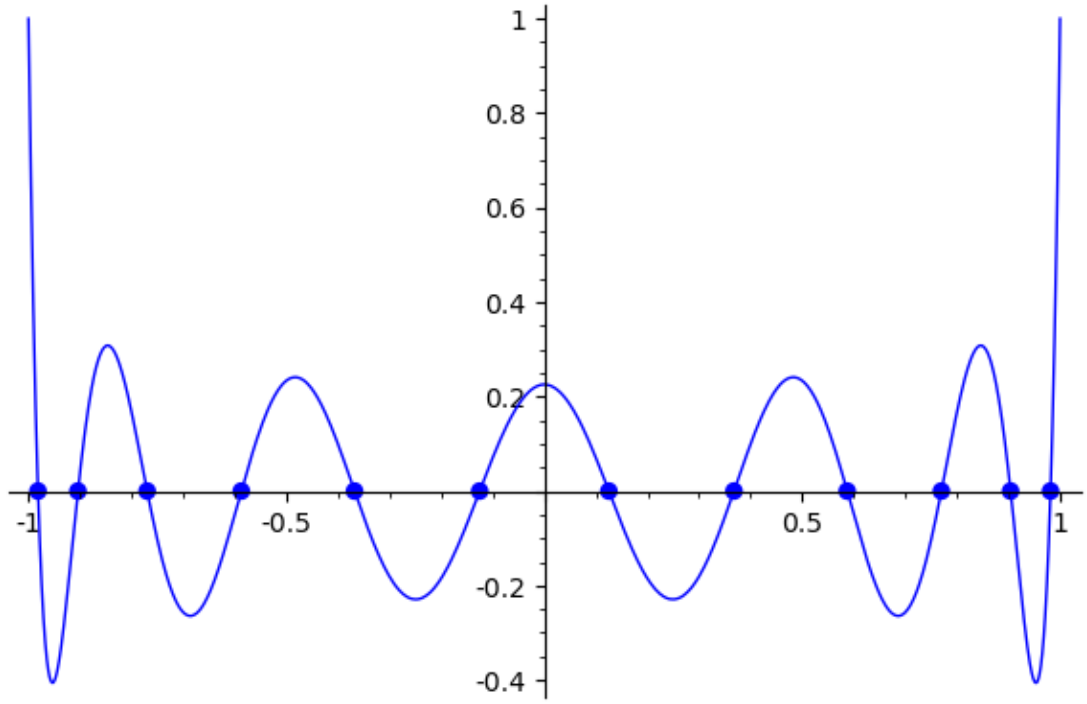
[4]:
```
# (a)
clear_vars()
x = var('x')

p = [gen_legendre_P(l, 0, x).plot((x, -1, 1),
color=random.choice(sorted(colors)),
legend_label=rf"$\ell={l}$") for l in range(0,6)]
plt = sum(p)
plt.legend(True)
show(plt)
```



[3]:

```
# (b)
clear_vars()
x = var('x')
poly = gen_legendre_P(12, 0, x)
p = plot(poly, (x,-1,1))
rts = poly.roots(ring=QQbar)
for rt in rts:
show(rt[0])
p = p + plot(point((rt[0], 0), size=45))
show(p)
```

```
-0.9815606342467193?

-0.904117256370475?

-0.769902674194305?

-0.5873179542866174?

-0.3678314989981802?

-0.1252334085114690?

0.1252334085114690?

0.3678314989981802?

0.5873179542866174?

0.769902674194305?

0.904117256370475?

0.9815606342467193?
```



[4]:
```
# (ci)
clear_vars()
x = var('x')
y = function('y')(x)
DE = (1-x^2)*diff(y, x, 2)-2*x*diff(y,x,1)+30*y==0

soln = desolve(DE, y, contrib_ode=True, show_method=True)[0][0]
show(soln.simplify_hypergeometric().simplify_full())
# Note the hypergeometrics. It seems simplify_hypergeometric does not␣
↪actually
# simplify hypergeometrics to special functions. gauss_a and gauss_b␣
↪are special cases of pFq
# hypergeomtrics and are an extension(?) of the Legendre polynomials, I␣
↪think. Getting this from Wikipedia
# and some SageMath documentation & forum threads.
```

```
y(x) == _K1*gauss_a(-5, 6, 1, -1/2*x + 1/2) + _K2*gauss_b(-5, 6, 1, -1/2*x + 1/2)
```

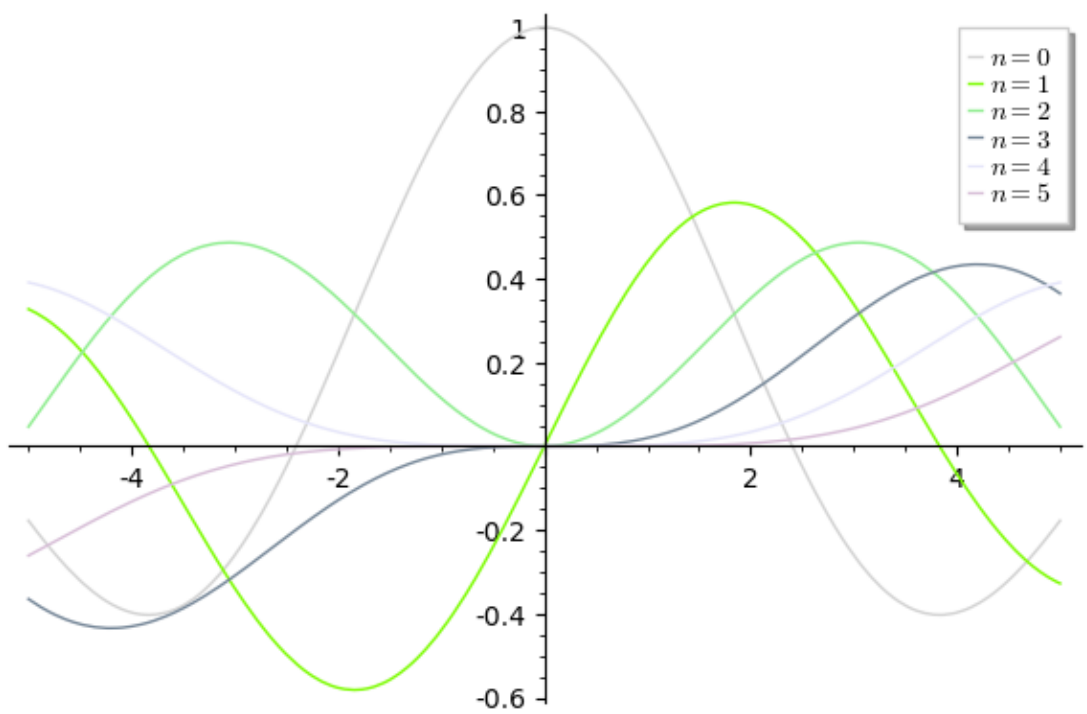
[5]:
```
# (cii)
clear_vars()
x = var('x')
y = function('y')(x)
```

```
DE = (1-x^2)*diff(y, x, 2)-2*x*diff(y,x,1)+(30-16/(1-x^2))*y==0

soln = desolve(DE, y, contrib_ode=True, show_method=True)[0][0]
show(soln)
```

```
y(x) == _K1*gauss_a(-9, 2, -3, -1/2*x + 1/2)/((x + 1)^2*(x - 1)^2) + _K2*gauss_b(-
9, 2, -3, -1/2*x + 1/2)/((x + 1)^2*(x - 1)^2)
```

[6]:

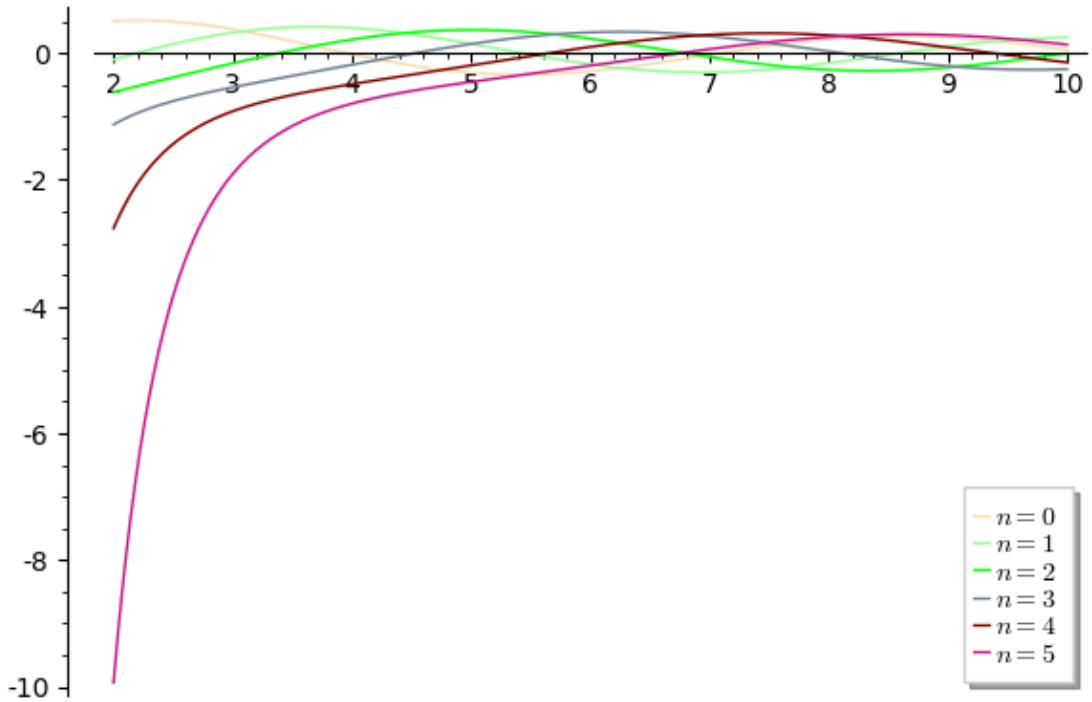
```
# (d)
clear_vars()
x = var('x')

p = [bessel_J(n, x).plot((x, -5, 5 ),
color=random.choice(sorted(colors)),
legend_label=rf"$n={n}$") for n in range(0,6)]
plt = sum(p)
plt.legend(True)
show(plt)
# We can see the expected damped oscillatory behaviour and boundedness.
```



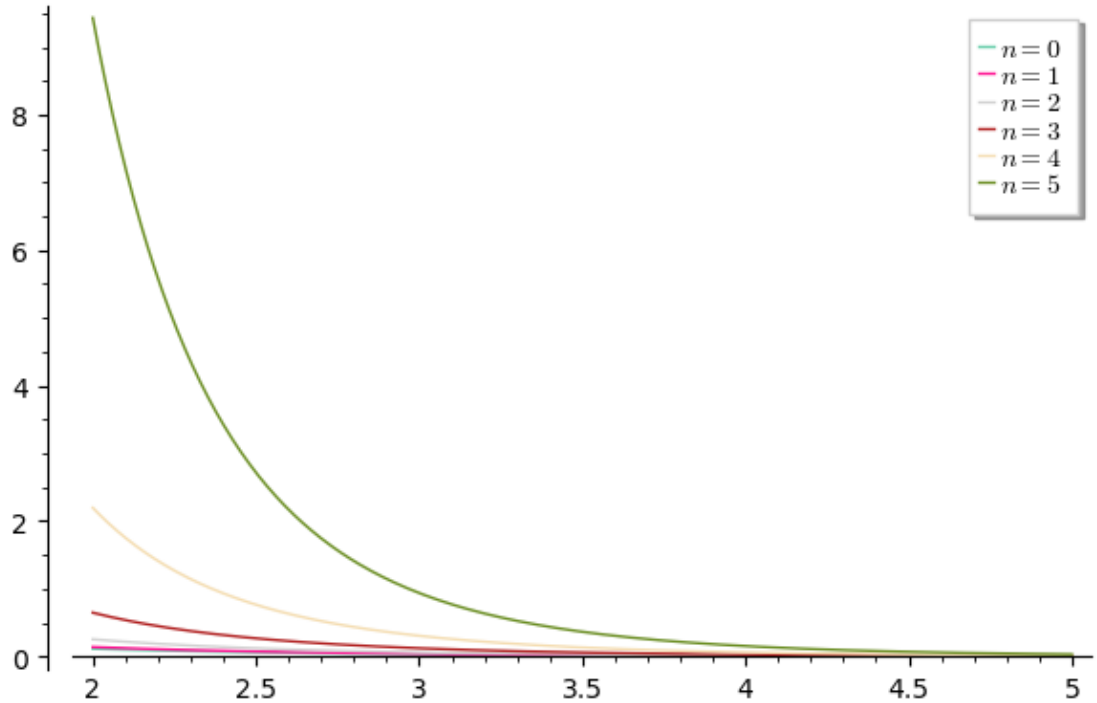
[8]:

```
# (e)
clear_vars()
x = var('x')
```

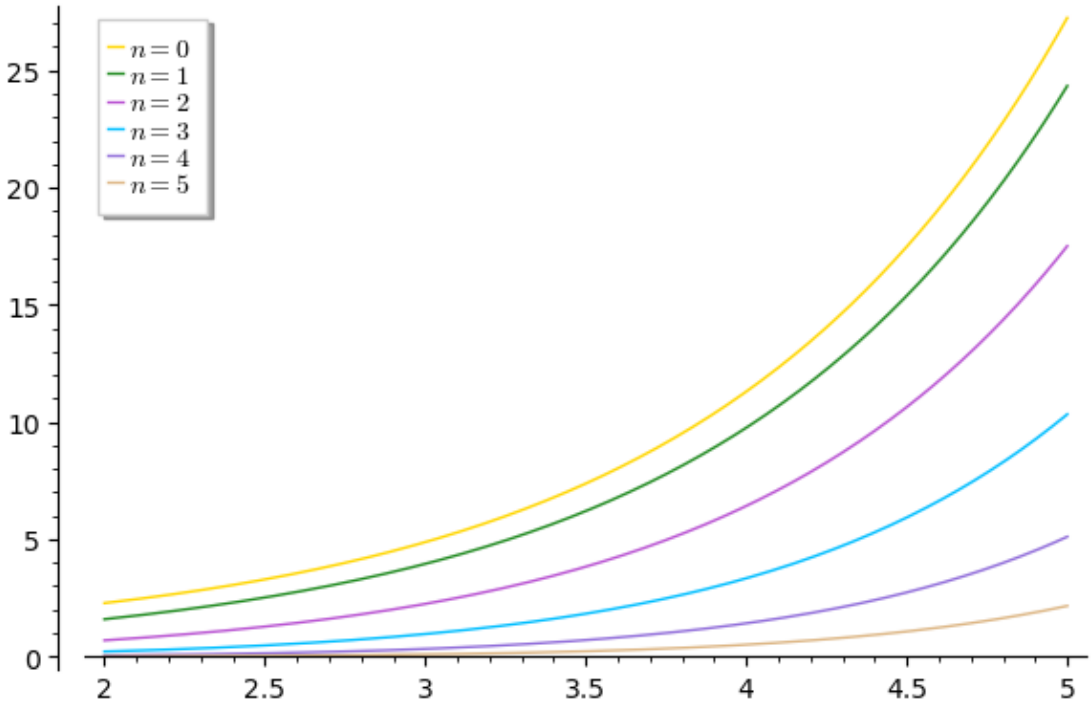
```
p = [bessel_Y(n, x).plot((x, 2, 10),
color=random.choice(sorted(colors)),
legend_label=rf"$n={n}$") for n in range(0,6)]
plt = sum(p)
plt.legend(True)
show(plt)
# We can see the damped oscillations and unboundedness near 0 requiring␣
↪its exclusion from many physical problems
```



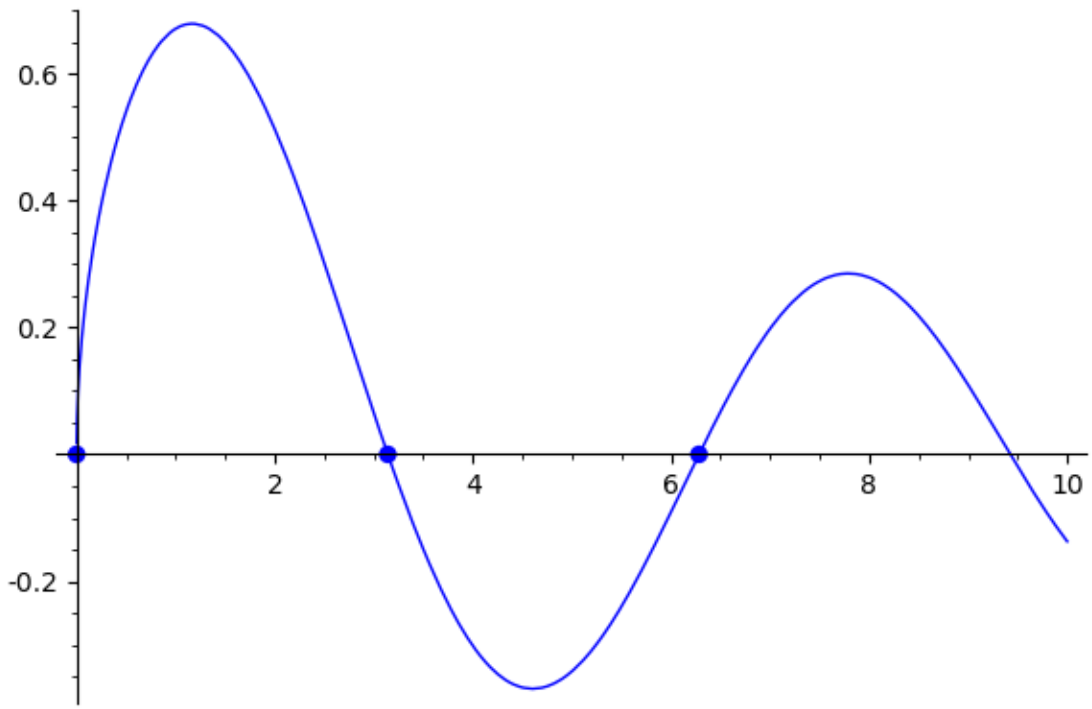
[9]:
```
# (f)
clear_vars()
x = var('x')

p = [bessel_K(n, x).plot((x, 2, 5),
color=random.choice(sorted(colors)),
legend_label=rf"$n={n}$") for n in range(0,6)]
plt = sum(p)
plt.legend(True)
show(plt)
# Again we can unboundedness near 0.
```



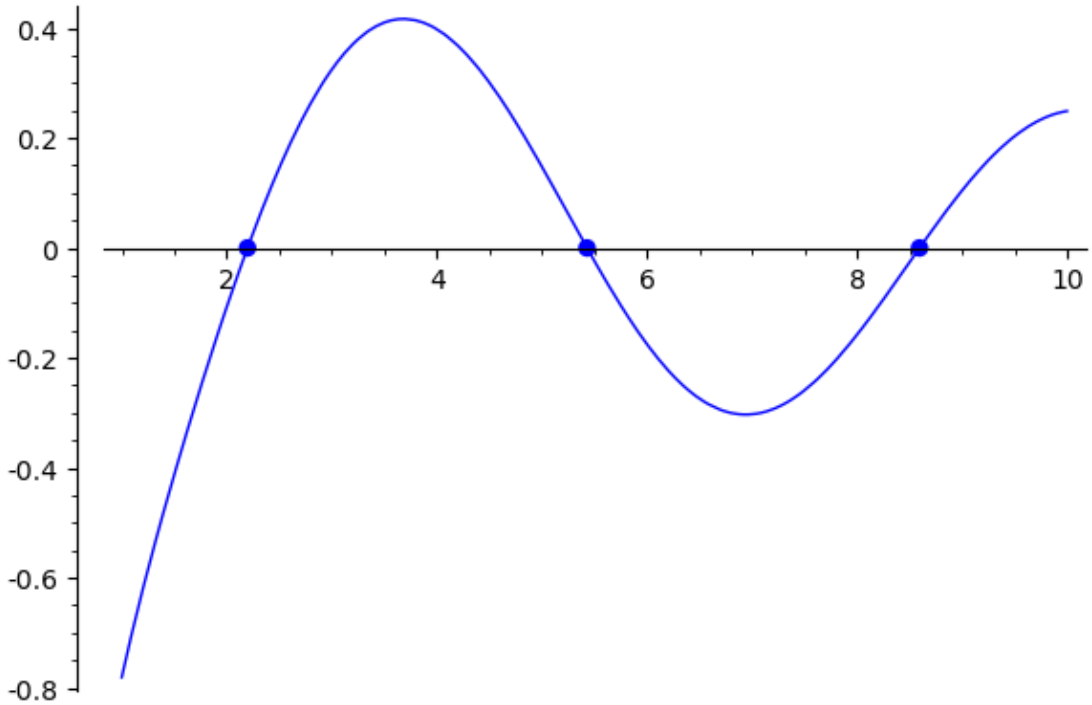
[10]:
```
# (g)
clear_vars()
x = var('x')

p = [bessel_I(n, x).plot((x, 2, 5),
color=random.choice(sorted(colors)),
legend_label=rf"$n={n}$") for n in range(0,6)]
plt = sum(p)
plt.legend(True)
show(plt)
# Here we see unboundedness for large x for a change.
```



[10]:
```
# (h)
clear_vars()
x = var('x')
poly = bessel_J(1/2, x)
p = plot(poly, (x,0,10))

for xv in [2,6]:
rt = find_root(poly, xv, xv+2)
p = p + plot(point((rt, 0), size=45))
p = p + plot(point((0, 0), size=45)) # Because there is a zero at zero␣
↪but the sage root finding tools don't play nice.
show(p)
```



[11]:
```
# (i)
clear_vars()
x = var('x')
poly = bessel_Y(1, x)
p = plot(poly, (x,1,10))

for xv in [2,5,8]:
rt = find_root(poly, xv, xv+1)
p = p + plot(point((rt, 0), size=45))
show(p)
```



[12]:
```
# (ji)
clear_vars()
x = var('x')
y = function('y')(x)
DE = (x^2)*diff(y, x, 2)+x*diff(y,x,1)-(4*x^2+9)*y==0

soln = desolve(DE, y, contrib_ode=True, show_method=True)
show(soln)
```

```
[[y(x) == I*_K1*bessel_I(3, -2*x) - _K2*bessel_Y(3, -2*I*x)], 'odelin']
```

[13]:
```
# (jii)
clear_vars()
x = var('x')
y = function('y')(x)
DE = (x^2)*diff(y, x, 2)+x*diff(y,x,1)+(4*x^2-9)*y==0

soln = desolve(DE, y, contrib_ode=True, show_method=True)
show(soln)
```

```
[[y(x) == -_K1*bessel_J(3, -2*x) - _K2*bessel_Y(3, -2*x)], 'odelin']
```

[14]:

```
        # (jiii)
        clear_vars()
        x = var('x')
        y = function('y')(x)
        DE = (x^2)*diff(y, x, 2)+2*x*diff(y,x,1)+(4*x^2-30)*y==0

        soln = desolve(DE, y, contrib_ode=True, show_method=True)
        show(soln)
        # Interestingly here Sage has opted to not go for Bessel functions␣
↪directly but instead their functional representations
        # in radicals, polynomials, and trig functions. I wonder why.
```

```
[[y(x) == 1/32*(2*(16/x - 420/x^3 + 945/x^5)*cos(2*x) - 15*(16/x^2 -
    112/x^4 + 63/x^6)*sin(2*x))*_K1*sqrt(-x)*sqrt(-x/pi)/x -
    1/32*(15*(16/x^2 - 112/x^4 + 63/x^6)*cos(2*x) + 2*(16/x - 420/x^3
    + 945/x^5)*sin(2*x))*_K2*sqrt(-x)*sqrt(-x/pi)/x],
'odelin']
```

[ ]:

```
```